Development of a Segmented, Reliable Communication Protocol for Data Transmission over Long-Range (LoRa) Links

Alessandro Monticelli

***Abstract*—This paper presents a lightweight transport protocol for point-to-point Long Range (LoRa©) networks to support the continuous transfer of large data blocks in emerging IoT applications. Existing fragmentation solutions typically rely on Stop-and-Wait ARQ schemes, incurring severe Time-on-Air overhead and rapid duty-cycle exhaustion. To overcome this, the proposed architecture divides payloads into up to 256 chunks per transmission window using a compact 7-byte header and per-chunk CRC-16. It offers dual operational modes: a best-effort streaming mode for high-frequency telemetry, and a reliable mode driven by a Selective Acknowledgment (SACK) bitmap. In reliable mode, SACK requires feedback only at the sequence boundary rather than per-packet, avoiding heavy Forward Error Correction overhead. We outline an experimental methodology evaluating the protocol across variable distances (1–5000 m) and payload sizes under Line-of-Sight and Non-Line-of-Sight conditions. Preliminary analyses show the SACK-driven approach significantly mitigates acknowledgment overhead and transmission latency, providing a flexible, high-capacity data link for resource-constrained embedded systems.**

***Index Terms*—**

I. Introduction

LOW-POWER, low-cost and long-range wireless communication technologies are becoming increasingly important in the context of the Internet of Things (IoT). Applications such as environmental monitoring, precision agriculture or Unmanned Aerial Vehicles (UAVs), require reliable data transmission over extended distances with strict power and hardware constraints [1].

A range of different communication standards, including ZigBee, Sigfox and LoRa© have been adopted to address these needs, and LoRa©, in particular, has emerged as the leading choice for low-power and long-range deployments thanks to its robust modulation technique and open network architecture [2]. However, conventional solutions based on LoRa© (particularly those following the Long Range Wide Area Network (LoRaWAN©) specifications) are optimized for small, non-frequent payloads [3].

This layout limitation poses severe restrictions on modern IoT applications, such as drone telemetry, environmental log collections, or remote imaging, which demand the continuous transfer of large data blocks [4], [5]. Standard LoRaWAN links require substantial transmission overhead, restrictive duty-cycle limitations, and lack optimized mechanisms for handling large, multi-packet payloads in point-to-point configurations.

This paper presents the design and implementation of a lightweight transport protocol for point-to-point LoRa© networks. Featuring a Selective Acknowledgment (SACK) scheme, a Hardware Abstraction Layer (HAL), and a dedicated reliable transmission mode, the proposed architecture efficiently supports the segmentation and reconstruction of large data payloads (up to approximately 62.7 KB). It enables robust data transfer over long distances with minimal radio duty-cycle footprint, operating entirely independently of higher-layer middleware (e.g., LoRaWAN©).

The key contributions of this work are three-fold:

- **High-Efficiency SACK Protocol**: Unlike existing LPWAN multimedia transmission protocols in literature that rely on Stop-and-Wait (S&W) ARQ, which requires an acknowledgment for every single packet and introduces severe latency and duty-cycle exhaustion, the proposed protocol implements a Selective Acknowledgment (SACK) scheme. By utilizing a compact, cumulative binary bitmap inside a single feedback packet, the receiver reports the status of up to 255 chunks in a single round-trip, minimizing over-the-air overhead and transceivers' duty-cycle footprint.
- **Lightweight Packet Segmentation and Abstraction**: The protocol introduces a 7-byte logical header and a custom CRC-16 integrity check on partial payloads, allowing robust reassembly of out-of-order chunks on resource-constrained microcontrollers.
- **Modular C++17 Implementation and TDD Validation**: The core state machine is completely decoupled from the physical radio driver through a Hardware Abstraction Layer (HAL). This design enables both native host-based unit testing (using the Unity framework) and physical deployment on Semtech SX126x transceivers, bridging the gap between simulation and real-world embedded systems.

This approach is generalizable to a wide range of IoT applications requiring extended data capacity over extended distances and high power efficiency, such as drone telemetry, marine biologging, or multi-agent coordinated communications.

II. State of The Art

Reliable packet transmission and fragmentation over Low-Power Wide-Area Networks (LPWANs) have been actively studied to support data-intensive IoT applications.

A prominent industrial standard is Static Context Header Compression (SCHC) [5], which introduces a framework for fragmenting IPv6 packets over resource-constrained networks. To guarantee reliability, SCHC ACK frames carry a bitmap representing the status of individual tiles within a window (ACK-on-Error). However, SCHC is designed as a generic adaptation layer for IP-based architectures, requiring pre-shared context rules and gateway-centric networks. Similarly, the LoRa Alliance standardized the Fragmented Data Block Transport specification to support Firmware Update Over The Air (FUOTA) using Forward Error Correction (FEC) codes to reconstruct blocks without feedback. However, FEC-based recovery introduces a constant over-the-air encoding overhead and high computational requirements for low-cost embedded systems.

For custom peer-to-peer LoRa networks, several proprietary and academic fragmentation protocols have been proposed. In [4], surveys on multimedia LoRa networks show that existing implementations primarily rely on Stop-and-Wait (S&W) ARQ schemes to recover lost packets (e.g., transmitting resized images in sequential segments). In Stop-and-Wait configurations, the transmitter blocks and waits for a dedicated acknowledgment frame for every single transmitted chunk. In high Time-on-Air (ToA) LoRa settings, S&W ARQ introduces severe idle latencies, keeps the half-duplex transceivers active for extended periods (draining battery), and quickly violates regulatory duty-cycle limitations.

In contrast, this paper proposes a transport-layer protocol that bridges these limits. By utilizing a Selective Acknowledgment (SACK) mechanism with a cumulative binary bitmap, the protocol enables bulk transmission of up to 255 chunks in a single window, requiring feedback only at the end of the sequence. This design combines the low overhead of P2P communications with the robustness of bit-vector acknowledgments.

To clearly position the proposed protocol within the current technological landscape, Table I provides an architectural comparison with other prominent fragmentation and transport solutions over LoRa.

III. Proposed System

The proposed protocol, `LoRaMultiPacket`, operates as a custom transport-layer protocol running directly over LoRa© physical layers. It aims to segment arbitrarily large payloads at the transmitter and reliably reconstruct them at the receiver, while keeping Logical Header and over-the-air (OTA) overhead minimal.

A. OTA Packet Structure

Each individual packet transmitted over the air is structured sequentially as a fixed 7-byte Logical Header, a variable-length Application Payload (up to 246 bytes), and a 2-byte CRC footer. Unused bytes at the end of the payload are omitted from over-the-air transmission to minimize airtime.

The exact byte alignment and offsets of the packet structure are presented in Table II.

The fields in the Logical Header are defined as follows:

- **`messageId` (2 bytes)**: Unique message sequence identifier. All fragments composing the same segmented message share this ID.
- **`totalChunks` (1 byte)**: The total count of chunks into which the original large payload is divided.
- **`chunkIndex` (1 byte)**: The 0-based index of this specific fragment (0 to `totalChunks` - 1).
- **`payloadSize` (1 byte)**: The count of valid data bytes contained in the payload section.
- **`flags` (1 byte)**: A control bitfield used to manage transmission state. Bit definitions are:
  - `PACKET_FLAG_SOM` (`0x01`): Start of Message (first chunk).
  - `PACKET_FLAG_EOM` (`0x02`): End of Message (last chunk).
  - `PACKET_FLAG_ACK_REQ` (`0x04`): Acknowledgment requested.
  - `PACKET_FLAG_ACK` (`0x08`): Acknowledgment feedback packet.
- **`protocolVersion` (1 byte)**: Identifies the version of the protocol to ensure backward compatibility.

After the payload, a 16-bit CRC-16 (Modbus CRC-16, polynomial `0xA001`, initialized to `0xFFFF`) is appended. The checksum covers the entire 7-byte header and only the valid `payloadSize` payload bytes, preventing transmission corruptions.

B. Selective Acknowledgment (SACK) Retransmission

To provide reliable delivery under lossy radio conditions while maintaining minimal airtime overhead, the protocol implements a Selective Acknowledgment (SACK) mechanism [6].

1) Retransmission Handshake: When a transmission is executed in reliable mode:

1) The transmitter segments the payload, serializes the chunks, and streams them sequentially. The final chunk index ($Index = \texttt{totalChunks} - 1$) is marked with the `PACKET_FLAG_ACK_REQ` flag.
2) Upon receipt of the chunk requesting acknowledgment, the receiver evaluates its reassembly session. It identifies which fragments are still missing by checking its internal bitmask of received chunks.
3) The receiver responds by transmitting a SACK packet (indicated by the `PACKET_FLAG_ACK` flag). The SACK's payload contains a binary bitmap where each bit represents the receipt status of a chunk (1 for received, 0 for missing).
4) The transmitter parses the received SACK bitmap. If some chunks are missing, it selectively retransmits only those lost chunks. The last retransmitted chunk has the `ACK_REQ` flag set again to query the receiver.
5) This selective retry loop continues until the full message is reconstructed at the receiver, or the transmitter reaches the maximum retry count.

2) Half-Duplex Hardware Switching Guard: Because LoRa transceivers are half-duplex, a hardware transition period is required for the transmitter to switch from transmit (TX) mode

TABLE I
ARCHITECTURAL COMPARISON OF LORA TRANSPORT SOLUTIONS

Feature	S&W	SCHC	LoRaWAN Frag.	Proposed
ACK Type	Per-packet	Bitmap	None (FEC)	Cumulative SACK
Overhead	Very High	Medium	High (FEC)	Very Low
Complexity	Low	High	High	Medium
Large Payloads	Poor	Good	Good	Excellent
Reliability	High	High	Statistical	High

Byte 0 – 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6
`messageId`	`totalChunks`	`chunkIndex`	`payloadSize`	`flags`	`protocolVersion`

Fig. 1. Byte layout of the 7-byte Logical Header.

TABLE II
LORAMULTIPACKET OTA PACKET LAYOUT AND OFFSETS

Offset (Bytes)	Field Name	Data Type	Size (Bytes)
0–1	`messageId`	uint16_t	2
2	`totalChunks`	uint8_t	1
3	`chunkIndex`	uint8_t	1
4	`payloadSize`	uint8_t	1
5	`flags`	uint8_t	1
6	`protocolVersion`	uint8_t	1
7	`payload`	uint8_t[]	N (up to 246)
$7 + N$	`crc`	uint16_t	2

to receive (RX) mode after sending a packet. If the receiver replies instantly, the transmitter will miss the SACK preamble. To guarantee correct reception, the receiver introduces a hardware guard delay of exactly 25 ms before transmitting the SACK packet.

3) Timeout and Fallback Recovery: If the transmitter does not receive a SACK packet within a timeout threshold of 1000 ms, it assumes that either the query or the response was lost. The transmitter falls back to retransmitting the final chunk of the sequence marked with `ACK_REQ` to trigger a new status report. Up to 5 consecutive timeouts are allowed before the transmission is aborted as failed.

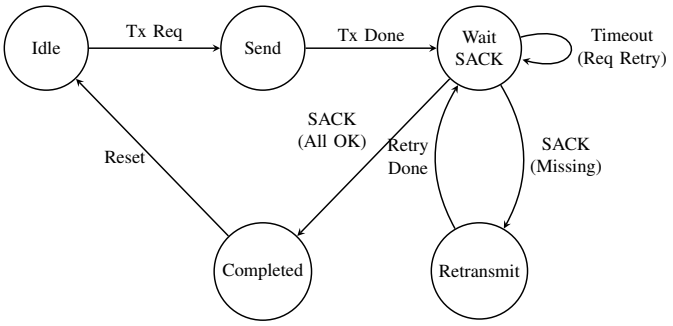

Fig. 2. State machine of the reliable transmission process at the transmitter.

IV. PRELIMINARY PROOF OF CONCEPT

An initial prototype used commercial Ebyte E220-series modules and validated the segmentation concept. That prototype demonstrated feasibility during flight tests but highlighted three important limitations: (1) the modules exposed a proprietary network layer and did not allow full control of the physical/link layer parameters, (2) repeating the full header in every chunk created measurable airtime overhead, and (3) there was no lightweight acknowledgment or retransmission mechanism.

To address these shortcomings, we designed the `LoRaMultiPacket` library on Semtech SX126x-class transceivers (demonstrated on a Heltec WiFi LoRa 32 V3 development board). The new implementation keeps the protocol independent of vendor-specific APIs and focuses on the primitives described above: a 7-byte header, a per-packet CRC-16 covering header and payload, and SACK-based reliable delivery. Receivers reassemble messages by matching the `messageId` and collecting fragments indexed from 0 to `totalChunks` - 1. Duplicate fragments are ignored by checking the received bitmap in the session.

V. HARDWARE LEVEL IMPLEMENTATION

To validate the proposed protocol, a complete implementation of the `LoRaMultiPacket` library was developed in C++17. The design is modular, separating the transport state machine from physical hardware dependencies.

A. Hardware Abstraction Layer (HAL)

To keep the protocol core hardware-agnostic, a clean Hardware Abstraction Layer interface (`RadioLibHal`) was implemented. On the physical nodes, the concrete class `EspHal` wraps the ESP32-S3 hardware resources:

- **SPI Communication**: Manages registers configuration and FIFO buffers transfer for the Semtech SX1262 transceiver.
- **GPIO Interrupts**: Maps physical pins, including Chip Select (NSS), Reset (RST), Busy status indicator (BUSY), and the hardware interrupt pin (DIO1) used to trigger RX/TX completion interrupts.
- **Timing Primitives**: Implements millisecond counters (`millis()`) and microsecond/millisecond delays (`delay()`).

B. Host-Based Unit Testing

Following a Test-Driven Development (TDD) methodology, the protocol core (e.g., packet parsing, reassembler sessions,

and CRC checksums) was validated in isolation on a desktop host (Linux/macOS). The tests compile and run natively using the **Unity** testing framework integrated into PlatformIO:

- **Integrity Tests**: Confirmed the Modbus CRC-16 computation on headers and partial/padded payloads.
- **Reassembly State Machine**: Validated reassembly handling of out-of-order and duplicate chunks.
- **Stale Session Pruning**: Confirmed that incomplete reassembly buffers are correctly pruned after exceeding the timeout threshold.

C. Physical Benchmarking Setup

Physical tests were conducted using two Heltec WiFi LoRa 32 V3 development boards. The transmitter node was connected to a console runner to sweep payload sizes, while the receiver node operated as a standalone listener. The transceivers were configured with the following parameters:

- **Carrier Frequency**: 868.0 MHz
- **Spreading Factor (SF)**: 7
- **Bandwidth (BW)**: 500.0 kHz
- **Coding Rate (CR)**: 4/5
- **Output Power**: 10 dBm
- **Preamble Length**: 8 symbols

VI. Proposed Experimental Methodology

To validate the reliability, latency, and throughput of the `LoRaMultiPacket` protocol, we outline a series of physical field experiments to be conducted under representative operational scenarios.

A. Experimental Parameters

The validation will sweep the following parameters:

- **Transmission Distance and Spatial Geometry**: Tests will be carried out at physical distances of 1, 10, 100, 250, 500, 1000, 2500, and 5000 meters. To account for spatial fading, antenna polarization discrepancies, and localized multi-path fading, testing in the short range (from 0 to 100 meters) will utilize a circular spatial sampling strategy, gathering data points at multiple angular offsets around the transmitting antenna. For long-range setups (greater than 100 meters) where circular navigation is impractical, data collection will follow a linear path.
- **Propagation Conditions**: All distance steps will be benchmarked under both Line-of-Sight (LoS) and Non-Line-of-Sight (NLoS) environments to test protocol performance under varying noise and packet corruption conditions.
- **Payload Sequence Size**: Payloads will scale from 1 chunk (representing the boundary condition) up to 100 chunks (approximately 24.6 KB total data size).
- **Radio Configuration**: Nodes will maintain standard settings (SF7, BW 500 kHz, CR 4/5, 10 dBm TX power) using the integrated Semtech transceivers.

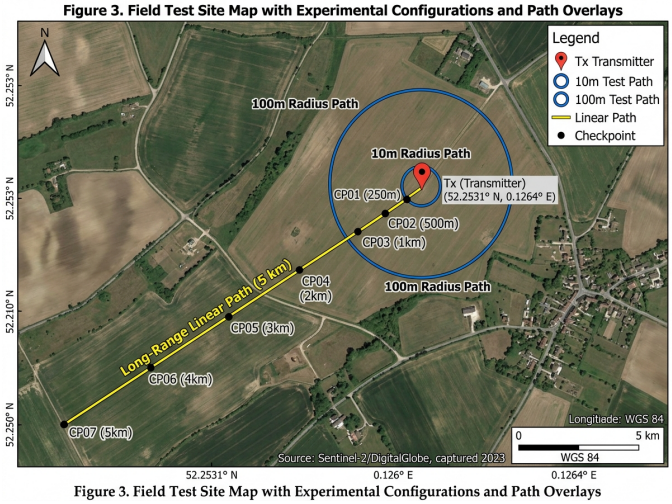

Fig. 3. Satellite view of the experimental testbed showing the transmitter node location, the circular spatial sampling points ($\leq$ 100 meters), and the linear range checkpoints ($>$ 100 meters) under Line-of-Sight (LoS) and Non-Line-of-Sight (NLoS) profiles.

B. Evaluation Metrics

For each combination of distance, environment, and payload size, 15 independent iterations will be performed. The following metrics will be collected and analyzed:

1) **Packet Delivery Ratio (PDR)**: The percentage of successfully reconstructed payloads at the receiver.
2) **End-to-End Latency**: The total duration elapsed from the initiation of split-packet serialization at the transmitter to the final payload reassembly callback at the receiver.
3) **Throughput (Goodput)**: The effective rate of application-level payload data delivered, calculated as $\text{PayloadSize (bits)}/\text{Latency (seconds)}$.
4) **Retransmission Overhead**: The count of extra chunks transmitted in reliable mode compared to the baseline case, representing the cost of SACK-driven error correction.

These benchmarks will allow us to evaluate the efficiency of the SACK retransmission handshake compared to best-effort unreliable streaming in lossy physical channels. The empty template for recording these empirical results is structured in Table III.

VII. Conclusion and Future Work

This paper presented the design and implementation of `LoRaMultiPacket`, a lightweight and reliable transport-layer protocol optimized for transmitting segmented large payloads over point-to-point LoRa links. By replacing traditional Stop-and-Wait per-packet acknowledgments with selective retransmissions requested via cumulative binary bitmaps, the protocol is expected to achieve high reliability under lossy conditions with minimal airtime overhead and duty-cycle footprint. The proposed testing methodology across varying physical distances and payload sizes will provide a comprehensive validation of the protocol's robustness for low-power, large-payload IoT communication scenarios.

TABLE III
LORAMULTIPACKET PLANNED EXPERIMENTAL BENCHMARKING RESULTS TEMPLATE

Distance (m)	Path Type	Payload Chunks	PDR (%)		Avg Latency (ms)		Avg Goodput (Kbps)		Retransmission Overhead	
			Case A	Case B	Case A	Case B	Case A	Case B	Chunks Sent	Retries
1	LoS	1–100								
	NLoS	1–100								
10	LoS	1–100								
	NLoS	1–100								
100	LoS	1–100								
	NLoS	1–100								
250	LoS	1–100								
	NLoS	1–100								
500	LoS	1–100								
	NLoS	1–100								
1000	LoS	1–100								
	NLoS	1–100								
2500	LoS	1–100								
	NLoS	1–100								
5000	LoS	1–100								
	NLoS	1–100								

For future work, several research directions will be explored to address open challenges in LPWAN communications:

- **Adaptive Spreading Factor Control**: Integrating an adaptive data rate (ADR) loop within the SACK handshake. By monitoring the retransmission count or packet loss rate from the binary bitmap feedback, the transmitter can dynamically adjust its Spreading Factor (SF) and Coding Rate (CR) to optimize transmission speed under clean channel conditions or improve resilience under deep fading.
- **Network Coexistence via CAD Backoff**: Stream-transmitting large packet sequences (up to 255 packets) consecutively can block shared channels. Future versions will introduce a Channel Activity Detection (CAD) check and a randomized backoff delay between subsequent chunks, enhancing network fairness and reducing collision probabilities in multi-node environments.
- **Hybrid FEC-ARQ Scheme**: Combining application-layer Forward Error Correction (e.g., erasure coding) with the SACK mechanism. The receiver will attempt to reconstruct lost packets using parity packets first, triggering the SACK feedback only when the number of lost packets exceeds the recovery threshold of the codes, further minimizing downlink acknowledgment traffic.
- **Windowed Selective Acknowledgment**: For very large payloads (exceeding 32 chunks), dividing the transmission into smaller sub-windows (e.g., 16 chunks per window) will allow the transmitter to query the receiver's state mid-transmission. This avoids wasting airtime and energy on transmitting subsequent chunks when a link drop occurs early in the sequence, while keeping the reassembly buffer size constrained.

APPENDIX A
TITLE OF APPENDIX ONE

Appendix one text goes here.

APPENDIX B

Appendix two text goes here.

ACKNOWLEDGMENT

The authors would like to thank...

REFERENCES

[1] M. Radeta, J. Pestana, P. Abreu, R. Freitas, F. Silva, D. Vieira, R. Prieto, M. Fernandez, F. Alves, T. Dellinger, S. Neves, and E. Delory, "Triton—open telemetry and location estimation for marine monitoring based on iot and lora," *IEEE Journal of Oceanic Engineering*, vol. 50, no. 2, pp. 1244–1258, 2025.

[2] T. Elshabrawy and J. Al-Ali, "Lora technology demystified: From link behavior to cell capacity," *IEEE Transactions on Wireless Communications*, vol. 17, no. 10, pp. 6611–6621, 2018.

[3] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melia-Segui, and T. Watteyne, "Understanding the limits of lorawan implementations," *IEEE Communications Magazine*, vol. 55, no. 9, pp. 34–40, 2017.

[4] S. De, H. M. Jalajamony, S. Adhinarayanan, S. Joshi, H. Upadhyay, and R. Fernandez, "Multimedia transmission over lora networks for iot applications: A survey of strategies, deployments, and open challenges," *Sensors*, vol. 25, no. 23, p. 7128, 2025.

[5] P. J. Marcelis, V. S. Rao, and R. V. Prasad, "Dare: Data recovery through application layer coding for lorawan," in *Proceedings of the 2017 IEEE/ACM Second International Conference on Internet-of-Things Design and Implementation (IoTDI)*, 2017.

[6] A. Calabrò, E. Marchetti, and M. T. Paratore, "Evaluating use of arq strategies in communication protocols for search and rescue," in *Proceedings of the 21st International Conference on Web Information Systems and Technologies (WEBIST)*. SCITEPRESS, 2025, pp. 59–70.

Alessandro Monticelli Biography text here.